

# Discrete element simulation of mill charge in 3D using the BLAZE-DEM GPU framework



Nicolin Govender<sup>a,c,\*</sup>, Raj K. Rajamani<sup>b</sup>, Schalk Kok<sup>a</sup>, Daniel N. Wilke<sup>a</sup>

<sup>a</sup> University of Pretoria, Department of Mechanical and Aeronautical Engineering, Pretoria 0001, South Africa

<sup>b</sup> Metallurgical Engineering Department, University of Utah, 135 South 1460 East, Salt Lake City, UT 84112, USA

<sup>c</sup> Advanced Mathematical Modeling CSIR, Pretoria 0001, South Africa

## ARTICLE INFO

### Article history:

Received 20 January 2015

Revised 15 April 2015

Accepted 22 May 2015

Available online 11 June 2015

### Keywords:

Large scale DEM

GPU

Tumbling mills

Ball mills

## ABSTRACT

The Discrete Element Method (DEM) simulation of charge motion in ball, semi-autogenous (SAG) and autogenous mills has advanced to a stage where the effects of lifter design, power draft and product size can be evaluated with sufficient accuracy using either two-dimensional (2D) or three-dimensional (3D) codes. While 2D codes may provide a reasonable profile of charge distribution in the mill there is a difference in power estimations as the anisotropic nature within the mill cannot be neglected. Thus 3D codes are preferred as they can provide a more accurate estimation of power draw and charge distribution. While 2D codes complete a typical industrial simulation in the order of hours, 3D codes require computing times in the order of days to weeks on a typical multi-threaded desktop computer. A newly developed and recently introduced 3D DEM simulation environment is BLAZE-DEM that utilizes the Graphical Processor Unit (GPU) via the NVIDIA CUDA programming model. Utilizing the parallelism of the GPU a 3D simulation of an industrial mill with four million particles takes 1 h to simulate one second (20 FPS) on a GTX 880 laptop GPU. This new performance level may allow 3D simulations to become a routine task for mill designers and researchers. This paper makes two notable extensions to the BLAZE-DEM environment. Firstly, the sphere-face contact is extended to include a GPU efficient sphere-edge contact strategy. Secondly, the world representation is extended by an efficient representation of convex geometrical primitives that can be combined to form non-convex world boundaries that drastically enhances the efficiency of particle world contact. In addition to these extensions this paper verifies and validates our GPU code by comparing charge profiles and power draw obtained using the CPU based code Millsoft and pilot scale experiments. Finally, we conclude with plant scale mill simulations.

© 2015 Elsevier Ltd. All rights reserved.

## 1. Introduction

### 1.1. Background and motivation

Since the first application of the discrete element method (DEM) for the simulation of grinding mills by Mishra and Rajamani (1990) in 1990 there has been a phenomenal growth in the variety of ways this technique is used in the mining industry. Prior to DEM, Powell's (1991) single ball trajectory in rotary mills was a key advancement in understanding lifter relief angle on the trajectory of charge. This approach continues to serve the mining industry even today.

In the late 90s two-dimensional DEM codes were the norm, due to the ease of execution on a personal computer with a single central processing unit (CPU). On the other hand, three-dimensional simulations promise greater accuracy of simulated results at the expense of computing time. At the outset it is useful to discuss the merits of 3D codes in comparison to 2D codes. The 2D code executes in a matter of hours on a CPU. It has been heavily used in hundreds of mining operations for annual or semiannual replacement of shell lifters (Dennis and Rajamani, 2001). This code has impacted the production, capacity and liner life of ball mills, autogenous mills and semi-autogenous (SAG) mills. The three-dimensional simulations are more accurate because the momentum transfer between balls and rock particles in the axial direction of the mill is accounted for. This opens the door for new insights when utilizing large-scale 3D simulations. 3D simulations has not been readily available to researchers and mill

\* Corresponding author at: University of Pretoria, Department of Mechanical and Aeronautical Engineering, Pretoria 0001, South Africa.

E-mail address: [govender.nicolin@gmail.com](mailto:govender.nicolin@gmail.com) (N. Govender).

designers since execution times are of the order of weeks on a single CPU for a typical plant size mill. This then becomes impractical to pursue on a routine basis.

Regardless of the severe computational burden of 3D simulations, a number of successes have been reported. In 2001 Venugopal and Rajamani (2001) presented a 3D DEM computational framework and compared it against the power draft in a laboratory scale 90 cm diameter mill. In the same year Mishra and Rajamani (2001) used the same 3D code for the prediction of power draft in plant scale mills. Herbst and Nordell (2001) combined 3D DEM with smoothed particle hydrodynamics (SPH) and the finite element method (FEM) to simulate slurry and solid charge motion, ore particle breakage and liner wear. Cleary (2001) demonstrated the sensitivity of charge behavior and power draft of a 5 m diameter ball mill to liner geometry and charge composition using a 3D code. There are continued advances in the simulation of breakage and slurry flow incorporating all the details in three-dimensional simulations. Morrison and Cleary (2008) describe the evolution of “Virtual Comminution Machine”, a simulation code that simulates breakage and slurry transport in tumbling mills. In their simulation both the discrete element method and smoothed particle hydrodynamics are employed for slurry and pebble flow through the grate slots and the pulp lifter. Cleary and Morrison (2009) show that 3D DEM combined with SPH is a viable tool for analyzing mineral processing equipment such as mills and twin deck screens. In a more recent study Alatalo and Tano (2010) compared the experimental deflection of a lifter in a ball mill with 3D predictions made with EDEM (Favier, 2014), a commercial DEM code. They concluded that 3D simulations agree better with predicted experimental values than 2D simulations. A full 3D simulation of a mill can provide valuable insights into the dynamics within a mill which can improve energy efficiency resulting in savings of thousands of dollars. For example demonstrate that the sparse packing formation in 2D has a significant effect on charge motion (see Fig. 30).

## 1.2. Computational aspects

An emerging trend of the past few years is the implementation of scientific and engineering solutions on a new class of processors termed General Purpose Graphical Processor Units (GPGPU) (Hromnik, 2013; Longmore et al., 2013; Harada, 2008), which offers CPU cluster computing performance at a fraction of the cost. Rajamani et al. (2011) shows a speed increase of up to 50x over CPU implementations for mill charge motion while Govender et al. (2014) showed a speed increase of up to 132x for polyhedral particles.

### 1.2.1. GPU hardware

The Graphic Processor Unit (GPU) was initially developed to reduce the computational burden on the CPU during the rendering process which involves the manipulation of millions of pixels on a screen simultaneously. This required that it be made up of mostly Arithmetic Logic Units (ALU) enabling it to perform these arithmetic operations in bulk. On the other hand, a CPU must perform complex control and logical operations that is needed to run an operating system in addition to arithmetic operations. A Streaming Multiprocessor (SM) on a GPU is equivalent to a core on the CPU. Each SM on a Kepler GK110 GPU can launch 2048 threads which are only capable of performing identical tasks (Single Instruction Multiple Data (SIMD)) (Sanders and Kandrot, 2010). Each CPU core is capable of launching two threads which can work independently and perform different tasks in each thread at higher clock speeds.

Fig. 1 illustrates the type of tasks that each unit excels at in computational performance. The limited GPU outperforms the versatile CPU when processing identical tasks. In order to realize a speed up on the GPU we need to ensure that our DEM algorithm is completely decoupled and expressed as SIMD tasks, in that we carry out the same instructions on different data elements which are particles in the case of tumbling mills.

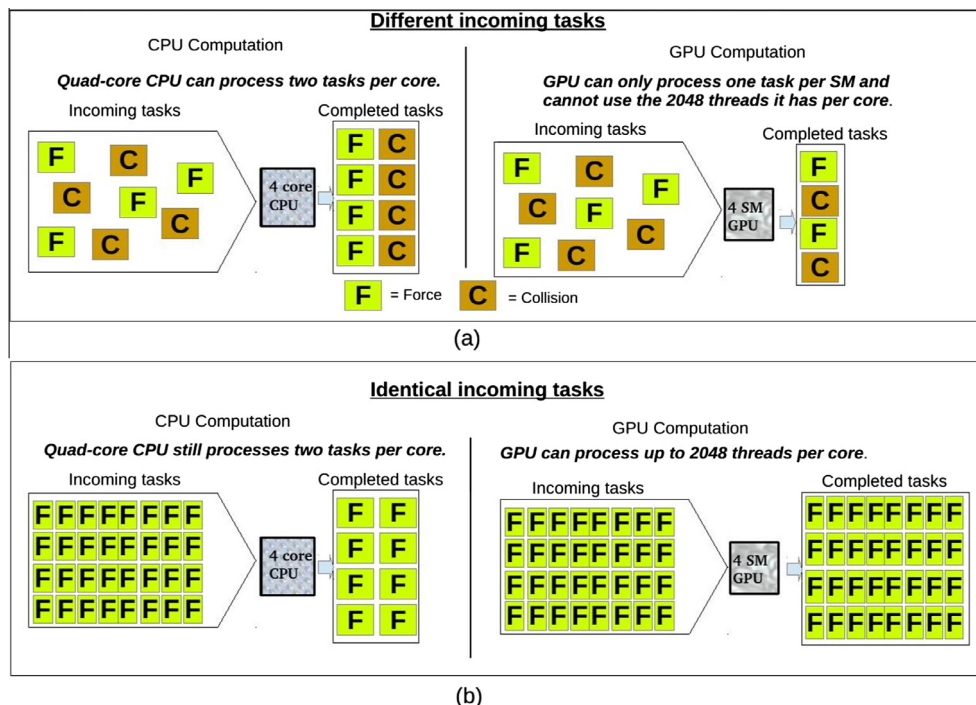


Fig. 1. Comparison between CPU and GPU task processing for the case of (a) different incoming tasks and (b) identical incoming tasks.

### 1.2.2. GPU software platform

The NVIDIA developed CUDA programming model (Sanders and Kandrot, 2010) provides access to the GPU from a variety of high level programming languages such as C++, Java and Python. CUDA batches threads into blocks (maximum 1024 threads) for execution on a SM. Threads within blocks can access fast shared memory with each thread in turn having access to its own 32 bit registers (fastest memory available). CUDA allows the creation of thousands of thread blocks containing millions of threads which get scheduled for execution on the hardware as SMs become available. Therefore, the algorithm must be independent of the execution order of the threads. The execution of a block is only completed once all threads within the block have reached an end point. This is very important and requires us to design algorithms that require nearly similar times to complete for all threads to best utilize the parallelism on the GPU. In this manuscript we show how plant scale mill simulations can be done in a matter of hours and within a day for very large mill simulations.

While most CPU DEM codes are now parallel, the CPU is still only a multi-core processor with the Intel Xeon chip having 12 cores on a single chip. Thus parallelism is limited to domain decomposition (Walther and Sbalzarini, 2009) as depicted in Fig. 2 (Cleary and Sawley, 1999). In a dynamic environment such as a tumbling mill particles move through multiple domains requiring interchange of data between processors which is time consuming. Furthermore particles are still processed in a serial loop on each CPU core. The GPU however is a many core processor enabling parallelism at a particle level, with each particle having its own thread.

Table 1 depicts the theoretical performance of a Xeon CPU and Tesla GPU for task of performing computations for 10 millions particles. In spite of the considerably higher clock rate the CPU can only launch 12 threads while the GPU can launch 53284 threads per cycle. This gives the GPU an enormous edge over the CPU in DEM calculations which are data parallel resulting in a speed up of 500 when taking into consideration cost and power consumption. Furthermore the thread scheduling is done automatically on the GPU by CUDA which removes the burden of ensuring effective

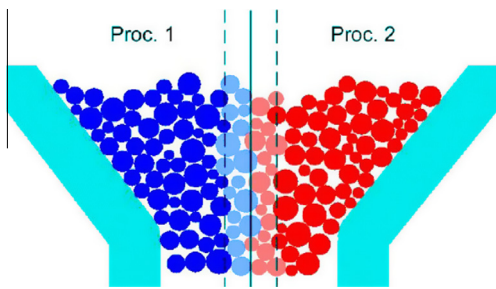


Fig. 2. Domain decomposition on CPUs (Cleary and Sawley, 1999).

**Table 1**  
Theoretical of CPU and GPU parallel solutions.

| CPU                                                                                                                                | GPU                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Intel® Xeon® Processor E7-8857<br>3.0 GHZ × 12 cores                                                                               | NVIDIA® Tesla® K80<br>1.0 GHZ × 26 SM 53,284 threads                 |
| Typical compute time for 10 <sup>6</sup> particles<br>Subdomains: 10 <sup>6</sup> /12 = 83 × 10 <sup>5</sup><br>particles per core | 10 <sup>6</sup> threads are created (one per particle)               |
| 3 computations per cycle (3 GHZ)                                                                                                   | 1 computations per cycle (1GHZ)                                      |
| Time for 10 <sup>6</sup> particles = $\frac{83 \times 10^5}{3} = 27,777$ s                                                         | Time for 10 <sup>6</sup> particles = $\frac{10^6}{53,284} = 18.76$ s |

parallel computation from the programmer. This also makes the same code scalable on future GPU hardware thus increasing performance without an change in code.

### 1.3. Discrete element method

The details of discrete element calculations are well documented in literature (Harada, 2008; Rajamani et al., 2011; Cundall, 1988; Cundall and Strack, 1979; Mishra and Rajamani, 1994). Here, we describe the computational steps necessary to implement our algorithm on the GPU. The flow-diagram in Fig. 3 describes the DEM process that we model (Hromnik, 2013; Govender et al., 2014). The bulk of computational time is spent on tasks 1 and 2 which is discussed in Section 1.4. Once contacting particles are determined, forces are calculated based on a force model as described in Section 1.5. We use the explicit velocity Verlet algorithm, which is second order accurate, to obtain the position  $\mathbf{x}$  and velocity  $\mathbf{v}$  of a particle  $i$  at time  $k$ :

$$\mathbf{x}_k^i = \left[ \mathbf{x}_{k-1}^i + \mathbf{v}_{k-1}^i \Delta t + \frac{1}{2} \mathbf{a}_{k-1}^i \Delta t^2 \right] \quad (1)$$

$$\mathbf{v}_k = \left[ \mathbf{v}_{k-1}^i + \frac{1}{2} (\mathbf{a}_{k-1}^i + \mathbf{a}_k^i) \Delta t \right] \quad (2)$$

The acceleration  $\mathbf{a}$  at time  $k$  is given by  $\mathbf{a}_k = \frac{\mathbf{F}_k^{\text{net}}}{m}$  where  $\mathbf{F}_k^{\text{net}} = \sum_{j=1}^L \mathbf{F}_k^{ij}$  is the sum of all  $L$  contact forces experienced by the particle. The angular velocity  $\omega$  of particle  $i$  at time  $k$  is obtained using the forward Euler integration scheme.

$$\omega_k = \omega_{k-1} + \alpha_k^{\text{ang}} \Delta t. \quad (3)$$

The angular acceleration  $\alpha^{\text{ang}}$  at time  $k$  is given by  $\alpha_k^{\text{ang}} = \frac{5}{2} r^2 m \Gamma_k^{\text{net}}$  where  $\Gamma_k^{\text{net}} = \sum_{j=1}^L \Gamma_k^{ij}$  is the sum of all  $L$  body contact torques (Eq. (10)) experienced by particle  $i$ ,  $r$  is the radius of the particle and  $m$  the mass.

An assumption in many DEM simulations is that particles are considered to be perfectly rigid for the duration of collision contact. In reality perfectly rigid particles do not exist, as all bodies will experience (to some extent) local deformations during contact. These deformations however occur on a time scale which is much smaller than what is required for capturing the macroscopic behavior of a system. Thus it is often sufficient to use a constitutive law, such as a linear spring (Cundall and Strack, 1979), to model

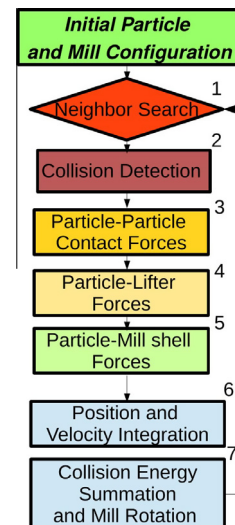


Fig. 3. Flow chart of DEM simulation procedure.

contact forces. Computing the time evolution of the system requires us to solve simultaneously Newton's equations of motion for all contacting particles, which on current hardware (2014) is only possible for a few thousand rigid bodies (Harada, 2008). Hence, we assume that there are only binary contacts (Zhao et al., 2006) between particles at any given time. The total force acting on a particle is obtained by summing the individual contributions of all the binary contacts of a particle per time step. This is a good approximation for many realistic applications that are not densely packed, provided the particles are of a similar size and move very little during a time step. Hence, computations can be carried out in parallel for all particles independent of each other.

In summary the assumptions that we make are:

1. Rigid bodies with single point contact.
2. Explicit integration.
3. Non-incremental friction model and
4. Similar particle sizes.

These assumptions result in a system that is completely decoupled and can be expressed as a Lagrangian type process. We are thus able to simulate the motion of individual particles independently of each other (Cundall and Strack, 1979). This results in the DEM being ideally suited to the GPU. Without these assumptions large scale DEM simulations would not be computationally feasible. The significant difference between current large scale CPU DEM simulations (Mishra and Rajamani, 1994; Langston, 2004; Jaelee, 2014; Cleary and Sawley, 2002) and our GPU implementation is the non-incremental friction model. Thus for simulations where the bulk behavior of a system consisting of millions of particles is required this compromise on the physics by using a simpler frictional model results in significant speed-up on the GPU.

#### 1.4. Additions to BLAZE-DEM framework for mill simulations

The mill simulation code used in this paper is built on the BLAZE-DEM GPU framework developed by Govender et al. (2014). The framework and its associated algorithms Govender et al. (2014) had been developed for solving dynamics problems involving particles within a hopper (Mack et al., 2011) or box. While the geometry of the volume within a hopper is convex resulting in only sphere-face contact as depicted in Fig. 4(a) the geometry of the volume within a mill however is not convex as depicted in Fig. 4(b). This results in sphere-edge contact in addition to sphere-face contact. Thus the first major addition to the BLAZE-DEM framework is an algorithm for the detection of contact between the particles and mill surfaces (world surfaces).

Current CPU and GPU DEM models represent the lifters and mill shell either as a collection of triangular surfaces or particles (Langston, 2004; Hromnik, 2013; Cleary, 2001) as depicted in Fig. 5(a). Apart from the large storage required, the computational cost associated with determining contact between a particle and the triangular mesh or surface particles is significant. We use a minimalistic approach of representing geometry as planar surfaces or geometrical primitives such as cylinders where possible as depicted in Fig. 5(b).

We employ a ray-tracing approach that only requires a normal  $\mathbf{n}_{\text{face}}$  and point  $\mathbf{C}$  of a surface to determine if there is contact. The distance between a particle at position  $\mathbf{P}_{\text{com}}$  and a planar surface is given by:

$$d = (\mathbf{n}_{\text{face}} \cdot (\mathbf{P}_{\text{com}} - \mathbf{C})) \quad (4)$$

If  $(\delta = d - \mathbf{P}_{\text{radius}}) < 0$ , where  $\mathbf{P}_{\text{radius}}$  is the radius of the particle there is contact between the particle and planar surface. The contact point given by:

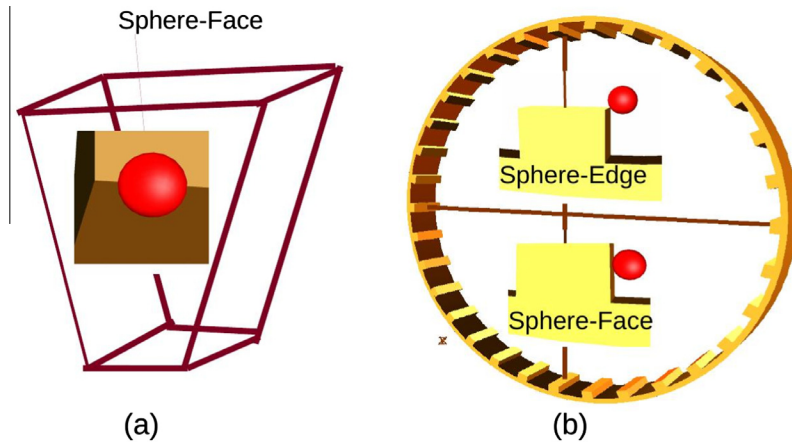


Fig. 4. Geometry specification for mills.

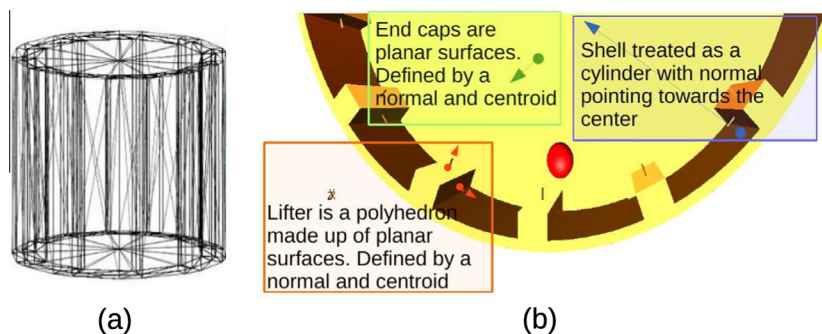


Fig. 5. Traditional geometry representations (a) and GPU representation (b).



$$\mathbf{P}_{\text{contact}} = \mathbf{P}_{\text{com}} + \mathbf{n}_{\text{face}} \delta \quad (5)$$

For a cylindrical object such as a mill shell we only need to consider plane sections perpendicular to the cylindrical axis, effectively making the Problem 2D. Thus the normal of a cylinder is given by the normalized vector  $\mathbf{P}_{\text{com}} - \mathbf{Cyl}_{\text{Center}}$  with the distance to the surface given by:

$$d = \|\mathbf{P}_{\text{com}} - \mathbf{Cyl}_{\text{Center}}\| - \text{Cyl}_{\text{radius}} \quad (6)$$

Using the above equations to determine contact for every particle is computational expensive but luckily only a fraction of the particles are in contact. Most approaches use a spatial grid approach algorithm that places the particles into cells (Hromnik, 2013; Harada, 2008; Zhao et al., 2006) with only neighboring cells being checked for contact. The BLAZE-DEM framework also uses such an approach that is optimized for the GPU (Govender et al., 2014), requiring 0.029 s to determine the nearest neighbors for a million particles on the GPU. Achieving such computational efficiency has a requirement that similarly sized particles be used (Munjiza and Andrews, 1998). However in ball mill simulations the mill surfaces also need to be checked for contact. To the best of the authors knowledge current DEM codes include the lifters in the spatial grid. Thus the minimum size of a cell is that of the largest lifter. This results in a much larger computational costs as there a large number of particles in a given cell which needs to be checked as depicted in Fig. 6(a). This approach is also restrictive in terms of mill geometry as discussed by Hromnik (2013). To maintain the computational efficiency of the BLAZE-DEM neighbor search algorithm we define the maximum cell size to be that of particle and employ heuristics to determine contact with the mill geometry as depicted in Fig. 6(b).

1. We firstly do a broad phase check if the particle is beyond the Bound Cylinder as depicted in Fig. 6(b).
2. If a particle passes this check we then loop over all lifters to check if there is intersection between the particle and the bounding cylinder with radius  $r_{\text{lifter}}$  of a lifter, as depicted in Fig. 6(b). If there is no intersection with any of the lifters then we check contact with the mill-shell using Eq. (6).

Heuristic 1 requires  $\mathcal{O}(N)$  computations and heuristic 2 requires  $\mathcal{O}(K)$  computations where  $K$  is the number of lifters and  $N$  is the number of particles. If there is an intersection between a ball and

a lifter we then find the contact normal and penetration distance as described in Algorithm 1.

#### Algorithm 1. Particle-lifter detailed collision detection.

##### 1. Loop over all lifter faces

- (a) Compute the distance using Eq. (4) between the lifter face and particle.
- (b) If this distance  $d$  is less than the particle radius we have possible contact.
  - i The contact point is given by Eq. (5).
  - ii Check if the contact point is actually on the face as Eq. (5) indicates contact for an infinite plane.
  - iii Check if the penetration  $\delta = d - r$  is valid (max 5% of radius) ( $\delta < 0.05r$ ).
  - iv If this is true there is contact at point  $\mathbf{P}_{\text{contact}}$  with normal  $\mathbf{n}_{\text{face}}$  and penetration distance  $\delta$ .

##### 2. If there is no contact with the faces then do a check for contact with edges. (This is computationally more expensive).

##### 3. Loop over all lifter edges

- (a) Compute the vector  $\mathbf{L}_{\text{PE}} = (\mathbf{P}_{\text{com}} - \mathbf{E}_i^0)$  that gives the distance between the particle and lifter edge, where  $\mathbf{E}_i^0$  is a vertex on the lifter edge.
- (b) Now check if this vector is valid. If  $(\mathbf{E}_i^{\text{Dir}} \cdot \mathbf{L}_{\text{PE}}) > 0$ , where  $\mathbf{E}_i^{\text{Dir}}$  is the direction along the lifter edge, then
  - i Compute the contact point  $\mathbf{P}_{\text{contact}} = (\mathbf{E}_i^0 + \|\mathbf{L}_{\text{PE}}\| \mathbf{E}_i^{\text{Dir}})$ .
  - ii Check if the distance  $d = \|\mathbf{P}_{\text{com}} - \mathbf{P}_{\text{contact}}\|$  between the point and the particle is less than the radius.
  - iii Check if the penetration  $\delta = d - r$  is valid (max 5% of radius) ( $\delta < 0.05r$ ).
  - iv If this is true there is contact at point  $\mathbf{P}_{\text{contact}}$  with normal  $\mathbf{n} = (\mathbf{P}_{\text{com}} - \mathbf{P}_{\text{contact}})/d$  and penetration distance  $\delta$ .

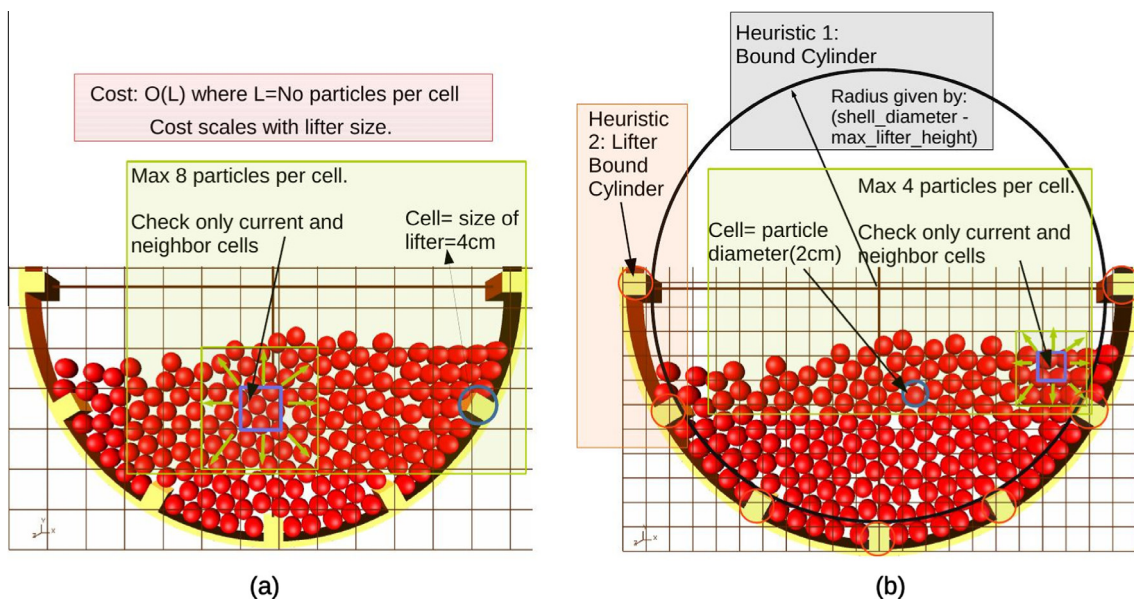


Fig. 6. (a) Traditional spatial grid representation and (b) BLAZE-DEM spatial grid representation.

The storage of geometrical information on the GPU has a major impact on performance as there is speed difference of 100X between the plentiful global ram (12 GB) and the much smaller constant memory (48 KB). However due to our minimalistic

```

/*-----*/
/* Dynamic World Object SIZE = 256B */
/*-----*/
struct DYOBJECT
{
    /*1. Bounding Object Information */
    uint   btype; /* Cylinder/Sphere */
    float3 Axis; /* Orientation ( along z axis) */
    float   boundR; /* Radius of encompassing cylinder */

    /*2. Polyhedron Geometric Information */
    float3 COM; /* Center of Mass */
    int    num_vertex; /* Number of vertex */
    float3 Local_Vertex [8]; /* Vertex relative to COM */

    /*3. Surface Geometric Information */
    int    num_surfaces; /* Number of planar surfaces */
    Surface surfaces [3]; /* Normal, Centroid etc */
    int    num_edges;
    EdgeD_Def edge[2]; /* Vertex that makes up the edge */
};
/*-----*/

```

Fig. 7. Lifter geometry storage on the GPU.

representation we can store up to 128 lifters which we classify as dynamic objects (Fig. 7) in the constant memory of the GPU in addition to the world and particle objects currently in constant memory (Govender et al., 2014).

In the BLAZE-DEM framework the geometry is static while for mill simulations the geometry is dynamic. Thus as the names suggests dynamic objects can be rotated and translated. Since constant memory can only be modified from the CPU the lifters are rotated on the CPU while GPU is busy with computations for the current step. We only need to rotate the vertices of a lifter as given by:

$$\text{vertex.x} = \cos(\theta) \times \text{refpoint.x} + \sin(\theta) \times \text{refpoint.y}$$

$$\text{vertex.y} = -\sin(\theta) \times \text{refpoint.x} + \cos(\theta) \times \text{refpoint.y}$$

$$\text{vertex} = \text{refpoint} + \text{Cyl}_{\text{Center}}$$

where  $\text{refpoint} = \text{vertex} - \text{Cyl}_{\text{Center}}$  and  $\theta$  is the angular displacement of the mill per step. Using the updated lifter vertex data the centroid and normals to faces are recalculated.

### 1.5. Force model

Fig. 8 shows the normal and tangential force models used in DEM simulations.

A linear spring dash-pot model is used to calculate the normal force between particles given by:

$$\mathbf{F}_N = (K_n \delta) \bar{\mathbf{n}} - C_n (\mathbf{V}_R \cdot \bar{\mathbf{n}}) \bar{\mathbf{n}}, \quad (7)$$

where  $\delta$  is the penetration depth,  $\mathbf{V}_R = \mathbf{V}_1 - \mathbf{V}_2$  is the relative translational velocity,  $K_n = \frac{m_{\text{eff}}}{t_{\text{contact}}^2} \ln(\epsilon)^2 + \pi^2$  is the spring stiffness,

$C_n = \frac{2 \ln(\epsilon) \sqrt{K_n m_{\text{eff}}}}{\sqrt{\ln(\epsilon)^2 + \pi^2}}$  is the viscous damping coefficient,  $\bar{\mathbf{n}}$  the normal

at contact,  $\epsilon$  is the coefficient of restitution and  $m_{\text{eff}} = \left(\frac{1}{m_1} + \frac{1}{m_2}\right)^{-1}$  is the effective mass of the particles. The contact time  $t_{\text{contact}}$  is determined by the properties of the material. However in most cases experimental data is not readily available for a particular material. For such cases  $K_n$  is chosen such that physical quantities of interest (such as energy) are conserved during integration for

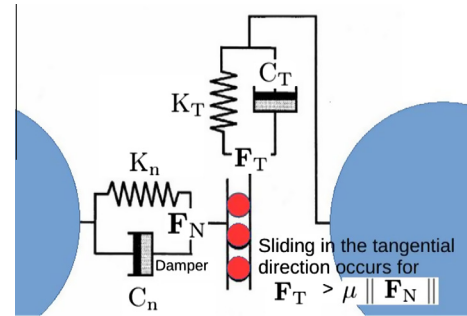


Fig. 8. Normal and tangential force models depicted by a spring dash-pot system.

the typical range of velocities observed in tumbling mill simulations (Venugopal and Rajamani, 2001; Hromnik, 2013; Mishra and Rajamani, 1994).

Typical DEM simulations use a spring stiffness and time-step that limits the maximum penetration depth to  $\delta_{\text{max}} \leq 0.05r$  where  $r$  is the radius of the smallest particle (Cleary, 2001; Longmore et al., 2013; Bell and Yu, 2005; Zhao et al., 2006).

**Tangential Contact.** In CPU DEM codes a linear spring dash-pot model is also used to calculate the tangential force given by:

$$F_T = -\min \left[ \mu \|\mathbf{F}_N\|, (K_T \int (\mathbf{V}_T dt) - C_T \|\mathbf{V}_T\|) \right], \quad (8)$$

where  $\mathbf{V}_T = (\mathbf{V}_R - (\mathbf{V}_R \cdot \bar{\mathbf{n}}) \bar{\mathbf{n}})$  is the relative tangential velocity,  $\mu$  the coefficient of friction,  $K_T$  the tangential spring stiffness and  $C_T = \frac{2 \ln(\epsilon) \sqrt{K_T m}}{\sqrt{\ln(\epsilon)^2 + \pi^2}}$  the tangential damping coefficient. The integral of

the tangential velocity over the duration of the contact behaves as an incremental spring that stores energy from the relative tangential motions between the particles. This represents the elastic tangential deformation of the contacting particles while the dash-pot dissipates a proportion of energy. The magnitude of the tangential force is limited by  $\mu \|\mathbf{F}_N\|$  at which point the particles begin to slide over each other.

However, due to the sorting of data on the GPU (Govender et al., 2014), maintaining contact history for the duration of the collision

is currently not possible to implement without severe performance penalties. Thus GPU codes use a history independent tangential model (Longmore et al., 2013; Govender et al., 2014; Bell and Yu, 2005) that ignores the elastic tangential deformation in the tangential direction during contact and only accounts for energy dissipation. This makes GPU computations much faster than CPU computations (Longmore et al., 2013; Govender et al., 2014; Neubauer and Radek, 2014). Therefore the GPU friction model is simplified to

$$F_T = -\min \left[ \mu \|\mathbf{F}_N\|, \frac{\min [\min [\|\mathbf{V}_{1T}\|, \|\mathbf{V}_{2T}\|], \mu \|\mathbf{V}_T\|] m_{\text{eff}}}{\Delta t} \right], \quad (9)$$

where  $\mathbf{V}_{1T}$  and  $\mathbf{V}_{2T}$  are the tangential velocities of each particle. This model has been shown to match experiments very well for simulations where the particles are constantly in motion in the previous works by the authors (Govender et al., 2014).

**Angular motion.** In addition to translation forces a particle also experiences a torque as a result of contact given by:

$$\Gamma = (\mathbf{r} \times \mathbf{F}) \quad (10)$$

where  $\mathbf{r}$  is the vector from the center of mass to the contact point  $\mathbf{PC}(x, y, z)$ .

#### 1.5.1. Calculation of power drawn by a mill

Grinding in the mineral processing industry is performed primarily by a ball mill which consumes a vast amount of energy and can account for as much as half of the processing cost. Thus understanding grinding mechanisms and estimating the power drawn by a mill can give guidance to improve the operational energy efficiency. The harsh environment inside the mill makes obtaining experimental data difficult. Thus the physical quantities calculated in a DEM simulation provide valuable insight to improve the efficiency of a mill. Since the power drawn by a mill is largely determined by the dynamics of the charge within the mill, we can obtain a good estimate of the power required by analyzing the energy loss mechanisms in a DEM simulation.

The total energy consumed by a mill is simply the net sum of the energy dissipated through contact  $E_{\text{diss}} = \sum_i^K (\|\mathbf{F}_{\text{diss}}\| \Delta x)$  where  $K$  is the number of contacts. Here  $\mathbf{F}_{\text{diss}}$  is given by the damping

and friction forces in Eqs. (7)–(9) respectively and is assumed to be constant over the distance  $\Delta x$  that the force acts.  $\Delta x$  is estimated by  $\|\mathbf{V}\| \Delta t$  since we do not store contact history in our GPU implementation. Thus the power consumed can be estimated by  $\text{Power} = \frac{E_{\text{diss}}}{t}$  where  $t$  is the duration over which we wish to calculate the power.

#### 1.6. Calibration of model parameters

In a DEM simulation the model parameters are chosen to either match experimental results or to reproduce a desired behavior. Tuning these parameters is a tedious task with the plethora of different models used in DEM simulations. Little guidance can be found in literature as the problems being simulated are quite often unique. In the area of ball mill simulations the 2D DEM code Millsoft developed by Mishra and Rajamani (1990) was the first code to be employed in the simulation of mills and has been validated extensively over the past two decades. Thus we verify our GPU code using a non-incremental tangential force model, Eq. (9), against Millsoft using an incremental tangential force model, Eq. (8). In mill simulations the dynamics of the system is governed by the rotation speed  $\Omega$  of the lifters. The distance covered by a lifter during a time-step is given by  $x_{\text{lifter}} = R \omega \Delta t$  where  $R$  is the radius of the mill and  $\omega = \frac{\pi}{30} \Omega$  is the angular velocity of the mill. Thus the maximum time-step for a mill rotating at  $\Omega$  rpm having a maximum penetration distance of  $\delta_{\text{max}}$  is given by:

$$\Delta t = \frac{\delta_{\text{max}}}{R \omega} \quad (11)$$

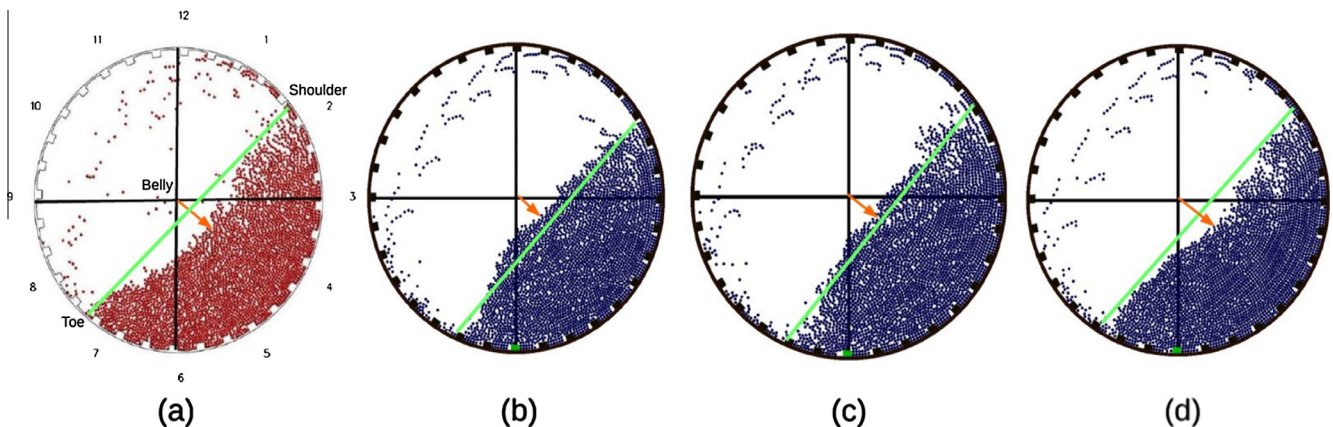
##### 1.6.1. Effect of parameters on charge profile

In this simulation we use a mill with diameter of 516 cm and length set equal to the ball diameter for simulating motion in 2D when using a 3D code. Thirty-two rows of rectangular lifters with a height of 9.5 cm and width of 15 cm is used with the mill rotating at 14 rpm. We attempt to tune the model parameters of the GPU code to reproduce the charge profile obtained using Millsoft. We start off by using the same parameters with the GPU code as given in Table 2. Note that the maximum allowed time-step given by Eq. (11) is  $\Delta t_{\text{max}} = 1.65 \times 10^{-5}$  using a maximum penetration distance of  $0.025r$ .

Fig. 9(a) shows the charge profile obtained with Millsoft and sub-figures (b)–(d) the GPU profiles for various values of  $\mu$ . We notice in Fig. 9(b) using the same frictional value that there is a good match with the shoulder position and the release point of the lifters. However there is a significant difference with the toe position in the GPU simulation at 6 30' while it is at 7 30' in the

**Table 2**  
Initial model parameters used in simulation for a 2D mill.

| Parameter | $K_n (\text{N m}^{-1})$ | $\epsilon$ | $K_T$           | $\mu$ | $\Delta t$         |
|-----------|-------------------------|------------|-----------------|-------|--------------------|
| GPU       | $4 \times 10^5$         | 0.45       | –               | 0.70  | $1 \times 10^{-5}$ |
| CPU       | $4 \times 10^5$         | 0.45       | $3 \times 10^5$ | 0.70  | $1 \times 10^{-5}$ |



**Fig. 9.** Charge profiles for CPU (a)  $\mu = 0.70$  and GPU, (b)  $\mu = 0.70$ , (c)  $\mu = 0.60$  and (d)  $\mu = 0.40$ ,  $N = 2916$  (radius = 2.5 cm).



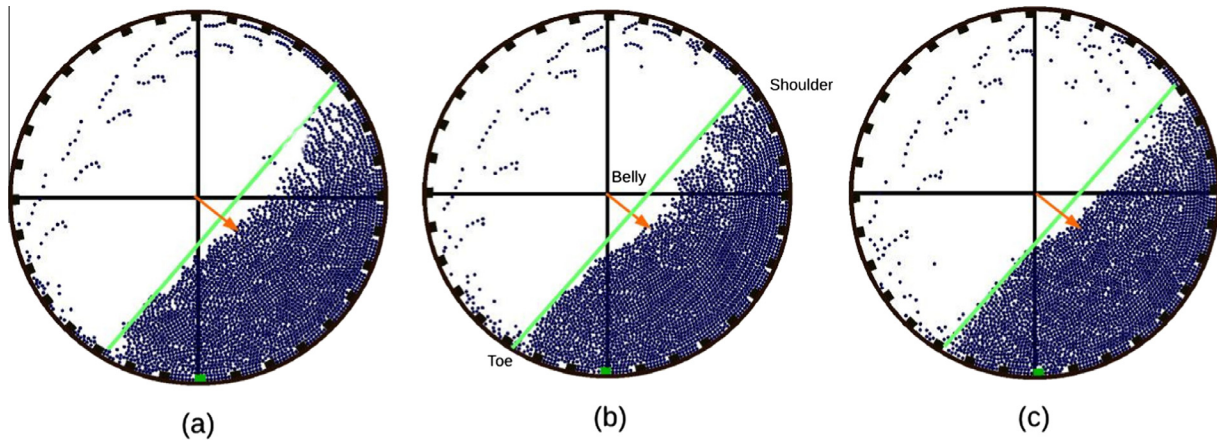


Fig. 10. GPU charge profiles for (a)  $\epsilon = 0.25$ , (b)  $\epsilon = 0.45$  and (c)  $\epsilon = 0.65$ ,  $N = 2916$  (radius = 2.5 cm).

CPU simulation. This difference is attributed to the absence of a restorative force in the GPU tangential model, which results in a greater resistive force being experienced by the particles for the same value of  $\mu$ . This keeps the center of mass of the distribution at a higher point as illustrated by the belly position. In Fig. 9(c) we use a lower friction value of  $\mu = 0.6$  in the GPU simulation. We notice that the toe is lower but the frictional force is still too high. In Fig. 9(d) we reduce the friction value to  $\mu = 0.4$  in the GPU simulation. We now see a much better agreement of the toe, shoulder and belly positions.

The second parameter that we can tune is the coefficient of restitution  $\epsilon$ . In Fig. 1 we vary the value of  $\epsilon$  while maintain the frictional value of  $\mu = 0.4$ . We notice that for  $\epsilon = 0.25$  (Fig. 10(a)) the distribution is packed much tighter as larger fraction of energy is lost during contact. For  $\epsilon = 0.65$  (Fig. 10(c)) the packing is less dense producing a greater dispersion when particles are released which provides a closer match to the distribution obtained with Millsoft (Fig. 10(a)).

Table 3  
Model parameters used in simulation for a 2D mill.

| Parameter | $K_n (\text{N m}^{-1})$ | $\epsilon$ | $K_T$           | $\mu$ | $\Delta t$         |
|-----------|-------------------------|------------|-----------------|-------|--------------------|
| GPU       | $4 \times 10^5$         | 0.65       | –               | 0.40  | $1 \times 10^{-5}$ |
| CPU       | $4 \times 10^5$         | 0.45       | $3 \times 10^5$ | 0.70  | $1 \times 10^{-5}$ |

Now that we have calibrated the GPU parameters, we vary the number of particles ( $N$ ) and compare the charge profiles using the same parameters as summarized in Table 3.

The charge profiles for  $N = 5344$  are compared in Fig. 11, while Fig. 12 compares the charge profiles for  $N = 11664$ . We notice a good match using the same set of parameters.

Fig. 13 shows sequentially the profile of charge ( $N = 2916$  particles) as the charge moves from rest to moving with the shell at the speed of the mill ( $3.82 \text{ m s}^{-1}$ ) and finally reaching a fully cascading motion. At steady state a slow moving region is created in the center of the charge.

## 2. Experimental validation of GPU DEM for mill simulations

### 2.1. Three-dimensional mill

We use the experimental data obtained by Venugopal and Rajamani, 2001 using a 90 cm diameter by 15.0 cm length mill containing eight 4.0 cm square lifters. The face of the mill was made of Plexiglas™ as to enable photographing of the tumbling charge, with the shell and lifters being made of steel (density of  $7800 \text{ kg m}^{-3}$ ). The mill was operated at 30%, 50% and 70% of critical speed for two levels of mill filling, 20% and 30% by volume respectively using steel balls with a radius of 2.5 cm. Note that the maximum allowed time-step given by Eq. (11) is  $\Delta t_{\max} = 4.14 \times 10^{-5}$  using a maximum penetration distance of  $0.025r$ .

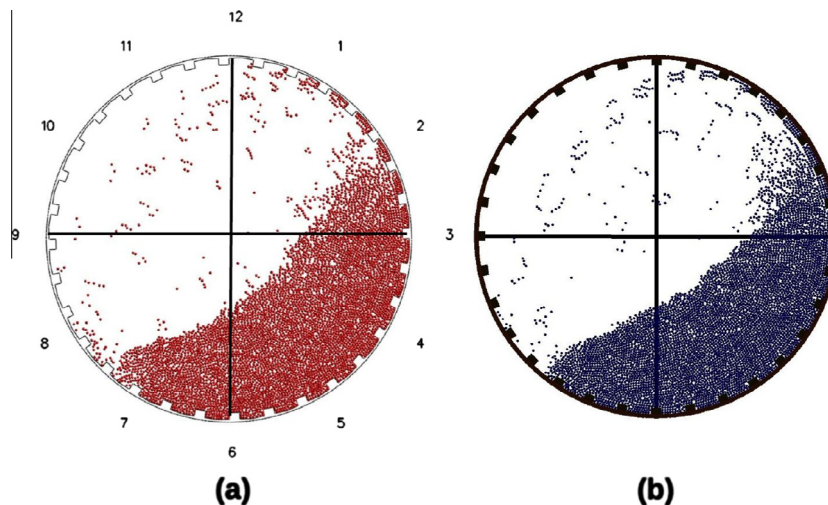


Fig. 11. (a) CPU and (b) GPU charge profiles.  $N = 5344$  (radius = 1.85 cm).



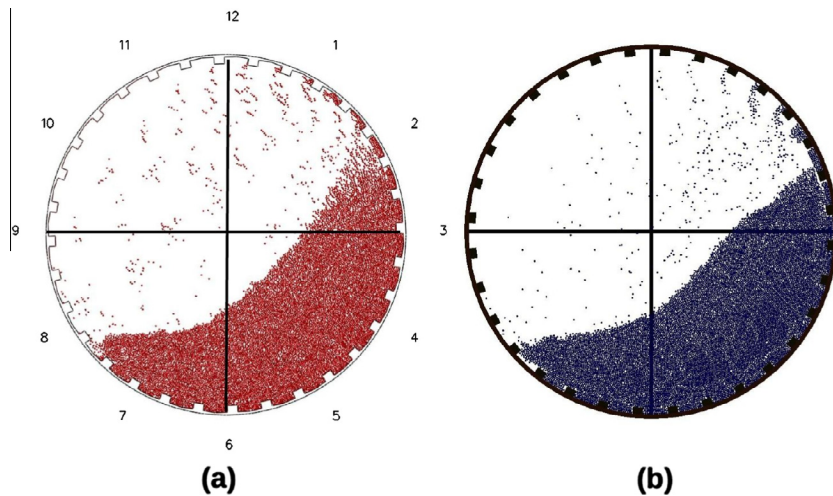


Fig. 12. (a) CPU and (b) GPU charge profiles.  $N = 11664$  (radius = 1.25 cm).

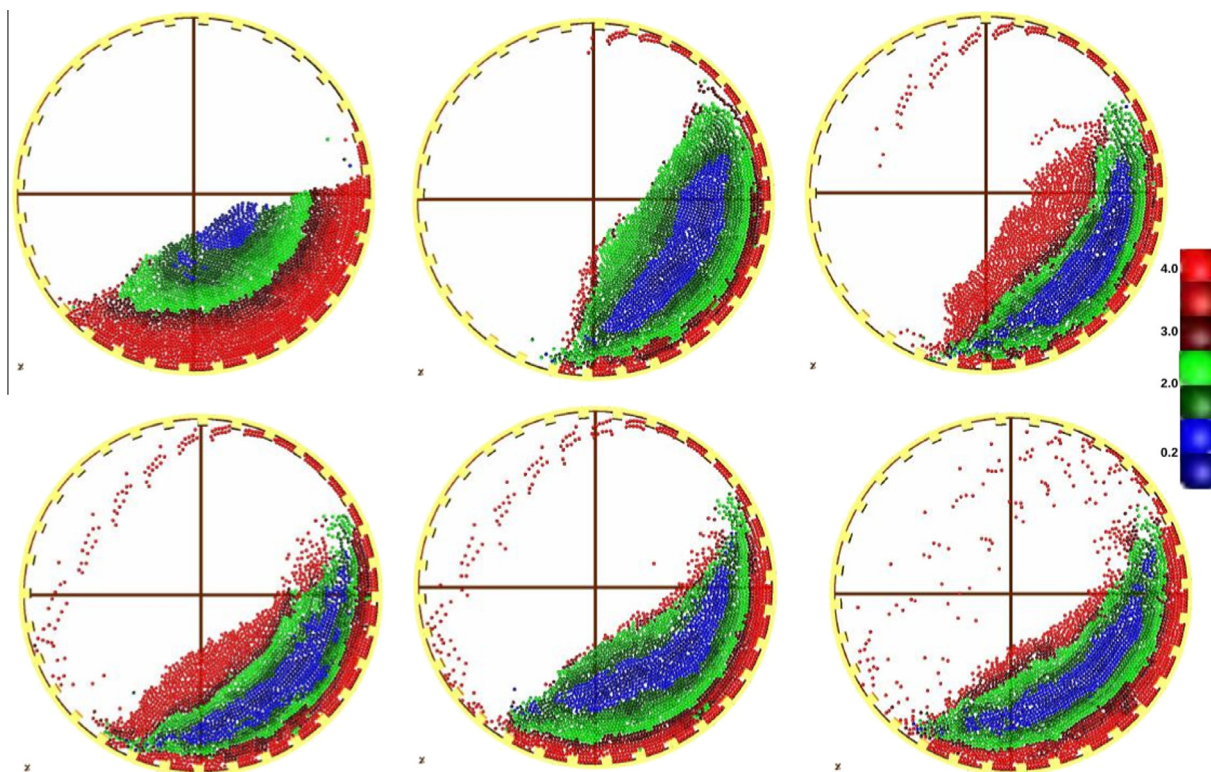


Fig. 13. (a) GPU velocity profile for  $N = 2916$  (radius = 2.5 cm).

#### 2.1.1. Calibration of model parameters

In the previous simulation the effect of parameters on the charge profile in the mill was studied. In this simulation we investigate the effect of the parameters on power draw as well for a mill with 20% loading at a mill speed of 32 rpm (70% of critical speed) drawing 532 W of power. We firstly match the charge profile by varying the frictional value as we saw in the previous simulation that this had the largest effect on the charge profile. Fig. 15 shows the charge profile for various values of  $\mu$ . We notice that the shoulder, toe and belly positions are lower as expected as we decrease the value of  $\mu$ . Fig. 14 (f) and (g) having values of  $\mu = 0.20$  and  $\mu = 0.15$  respectively gives the best match to the experimental profile depicted in Fig. 14(a). Thus the ideal value of  $\mu$  seems to be in the range of 0.15–0.20.

Fig. 15 depicts the charge profile for varying values of  $\epsilon$  using  $\mu = 0.20$ . We firstly notice that there is little change with the shoulder and toe positions with the belly becoming less dense as we increase  $\epsilon$ . The ideal value of  $\epsilon$  seems to be in the range of 0.80–0.85, as power at the limits of the range bounds the experimental power of 532 W.

Now that we have a reasonable match for the charge profile we tune the parameters to yield the desired power values. We use a finite number of combinations of  $\mu$  and  $\epsilon$  in the ranges that we deemed to yield the best charge profiles in Figs. 14 and 15. Table 4 contains the four combinations we use as well as the average and maximum errors obtained for mill speeds of 14, 22 and 32 rpm. Ideally the range of values should be more than the four we have chosen to get the best possible match. However we only

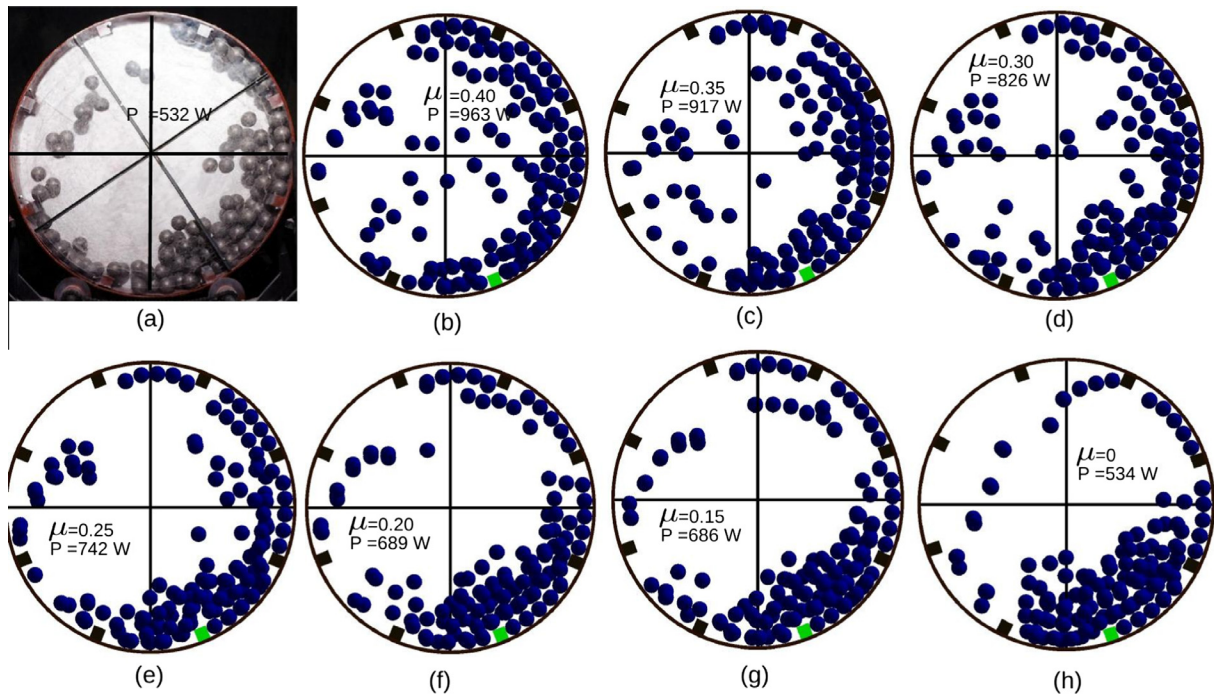


Fig. 14. (a) Experiment, (b) GPU charge profiles for different values of  $\mu$  as indicated.  $N = 168$  (20% filling),  $rpm = 32$  (70% critical speed).

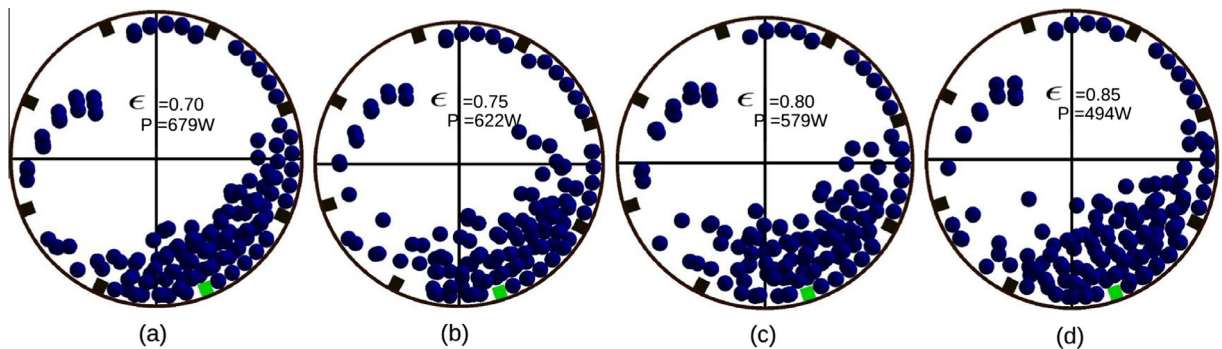


Fig. 15. GPU charge profiles for different values of  $\epsilon$  as indicated.  $N = 168$  (20% filling),  $rpm = 32$  (70% critical speed).

Table 4

Average error for various parameter combinations for a 3D mill.

| Combination | $\epsilon$ | $\mu_T$ | Max error (%) | Average error (%) |
|-------------|------------|---------|---------------|-------------------|
| 1           | 0.80       | 0.15    | 7.88          | 7.09              |
| 2           | 0.825      | 0.20    | 7.34          | 5.23              |
| 3           | 0.825      | 0.15    | 5.99          | 4.61              |
| 4           | 0.85       | 0.175   | 8.01          | 5.77              |

wish to show the effect of DEM parameters on mill simulations rather than do a detailed calibration of DEM parameters. Combination 3 gives the lowest error and will thus be used to predict power and charge profiles for the rest of this section.

### 2.1.2. Charge motion and power draw

Representative snapshots of the GPU DEM predicted charge profiles alongside the still camera images for each of the experiments are shown in Figs. 16–19 with the associated power draft in Table 4. Charge profiles predicted by the GPU DEM code are consistent with observed charge profiles. The positions of the toe and shoulder of the charge are also reasonable with a slightly lower toe

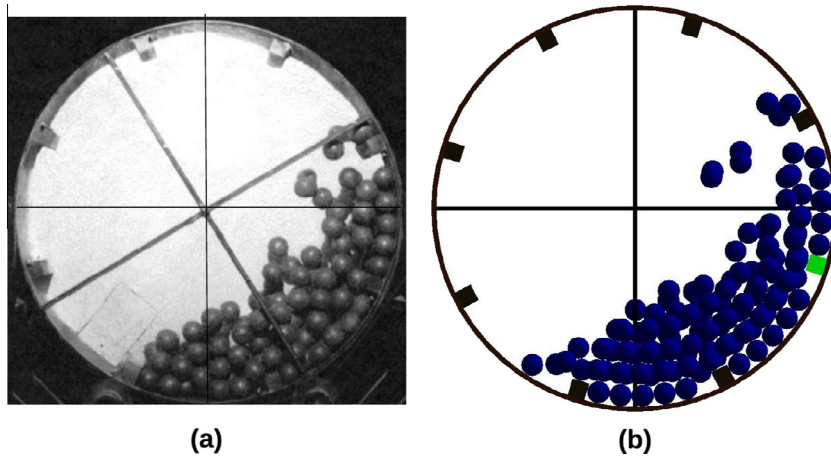
position and lower mass distribution which is expected due to the simpler tangential force model used. An exact match between charge trajectories is difficult to obtain due to the stochastic nature of the problem for slight deviations in the initial setup. In addition to the simplifying assumptions made in the DEM model as well as the mechanical losses and the geometry of the mill not being exactly the same due to wear and manufacturing processes.

Table 5 summarizes the computed power values. We see a good match for the 30% loading scenario as the error is only 4.07%. Note that we only tuned the parameters for the 20% loading scenario. This gives confidence in the selection of the model parameters as we can predict the effects of mill-load on the power draw.

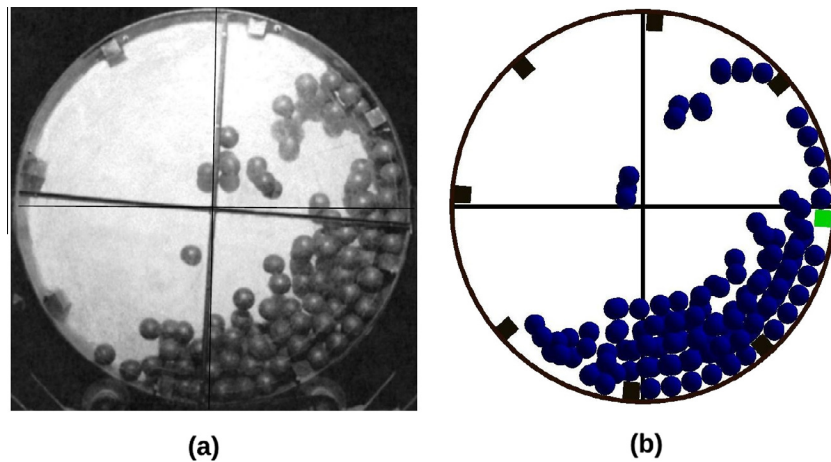
### 2.2. Charge motion and power draw for a slice mill

In this simulation we use the experimental data as given by Hlungwani et al., 2003 for a pilot mill with a diameter of 55 cm and length of 2.35 m containing twelve rows of 22 cm square lifters. The mill is loaded to 25% and 35% respectively with steel balls having a radius of 2.2 cm and density of  $7800 \text{ kg m}^{-3}$ . Thus this is a slice of a mill with a single layer ball charge moving between the steel shell and glass plate in the front. For their DEM simulations

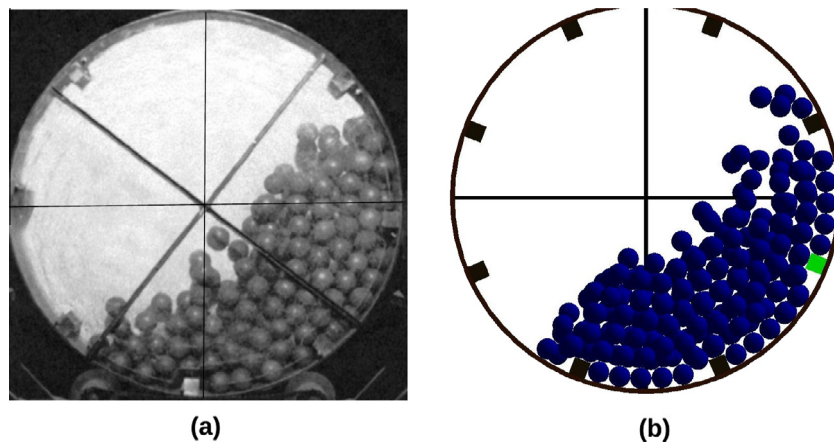




**Fig. 16.** (a) Experiment (Venugopal and Rajamani, 2001), (b) GPU charge profiles.  $N = 168$  (20% filling),  $rpm = 14$  (30% critical speed).



**Fig. 17.** (a) Experiment (Venugopal and Rajamani, 2001), (b) GPU charge profiles.  $N = 168$  (20% filling),  $rpm = 22$  (50% critical speed).



**Fig. 18.** (a) Experiment (Venugopal and Rajamani, 2001), (b) GPU charge profiles.  $N = 243$  (30% filling),  $rpm = 14$  (30% critical speed).

Moyes et al. use the same model parameters for different mill-speeds. However as discussed in Section 1.6 the model parameters (specifically the time-step) are determined by the rotation speed of the mill. The maximum allowed time-step using Eq. (11) for a maximum penetration distance of  $0.10r$  per step at

160% of critical speed (93.30 rpm) is  $\Delta t_{\max} = 4.14 \times 10^{-6}$ . To allow comparison with published results, we use the same  $\Delta t$  for increasing speed but also include results using a time-step given by Eq. (11) as mill speed increases. Table 6 summarizes the model parameters used.



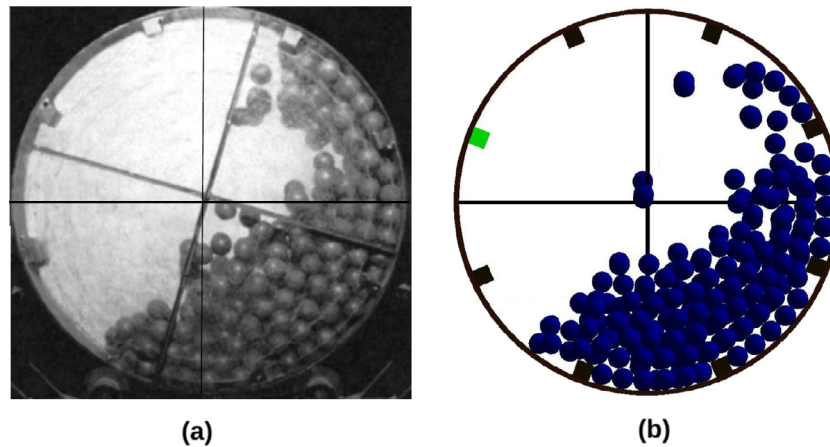


Fig. 19. (a) Experiment (Venugopal and Rajamani, 2001), (b) GPU charge profiles.  $N = 243$  (30% filling),  $rpm = 22$  (50% critical speed).

**Table 5**  
Power draw with experiment and GPU DEM for 3D mill.

| RPM | Filling (20%) |         | Filling (30%) |         |
|-----|---------------|---------|---------------|---------|
|     | Power (W)     |         |               |         |
|     | Experiment    | GPU DEM | Experiment    | GPU DEM |
| 14  | 301           | 336     | 393           | 409     |
| 22  | 459           | 437     | 617           | 636     |
| 32  | 532           | 520     |               |         |

Fig. 20 shows how the power varies as a function of rotation speed. We see a good match between both DEM codes and experiment for sub-critical speeds which is the normal operation mode of a mill. At super-critical speeds there is a slight difference due to CPU and GPU codes using a constant time step. However the results using the time step given by Eq. (4) as we increase mill

speed shows a better match to experiment. Note: There is a difference of 1.8 in computation time between the fixed and varied step GPU solutions.

Figs. 21 and 22 depict the charge profile for sub-critical speeds. We see a good match which is expected as the power values are similar. Note that it is difficult to do an accurate frame matching.

Figs. 23 and 24 depict the charge profile for super-critical speeds. We note that DEM correctly predicts 1 and 2 layers of centrifuging particles for 100 and 160 percent of critical speed respectively.

### 3. Industrial mill simulation

The Los Bronces SAG mill is a 10.12 m diameter by 4.7 m long mill rotating at 10 rpm. The mill charge is made up of 31% ore and 10% balls by volume. The power draft reported for this mill

**Table 6**  
Model parameters used in simulation for slice mill.

| Parameter | $K_n (\text{N m}^{-1})$ | $\epsilon_{\text{particle}}$ | $\epsilon_{\text{shell}}$ | $K_T$           | $\mu_{\text{particle}}$ | $\mu_{\text{shell}}$ | $\Delta t$         |
|-----------|-------------------------|------------------------------|---------------------------|-----------------|-------------------------|----------------------|--------------------|
| GPU       | $4 \times 10^5$         | 0.85                         | 0.80                      | –               | 0.15                    | 0.20                 | $2 \times 10^{-5}$ |
| CPU       | $4 \times 10^5$         | 0.66                         | 0.36                      | $4 \times 10^5$ | 0.14                    | 0.39                 | $2 \times 10^{-5}$ |

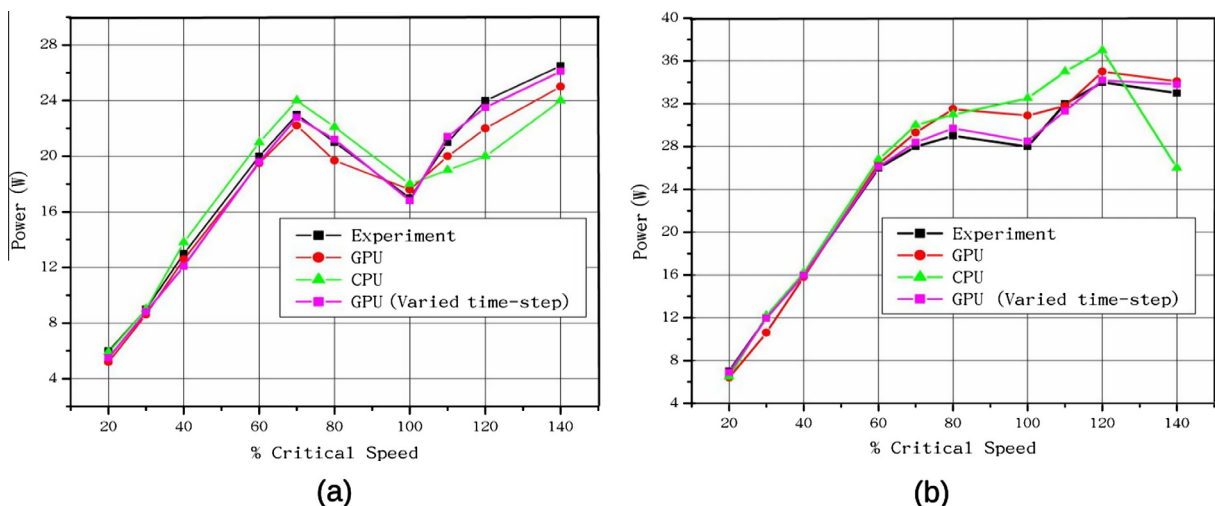
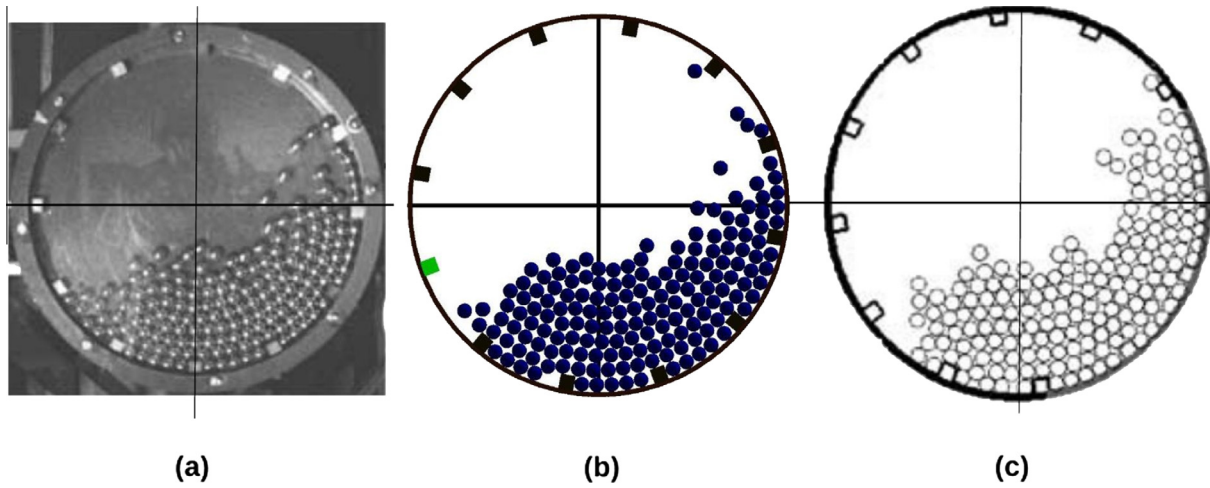
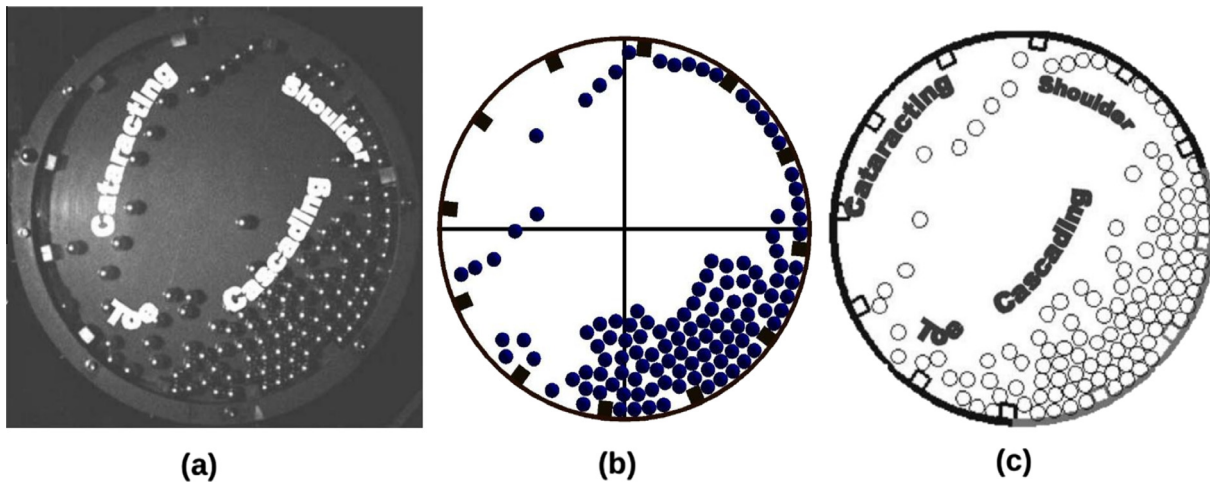


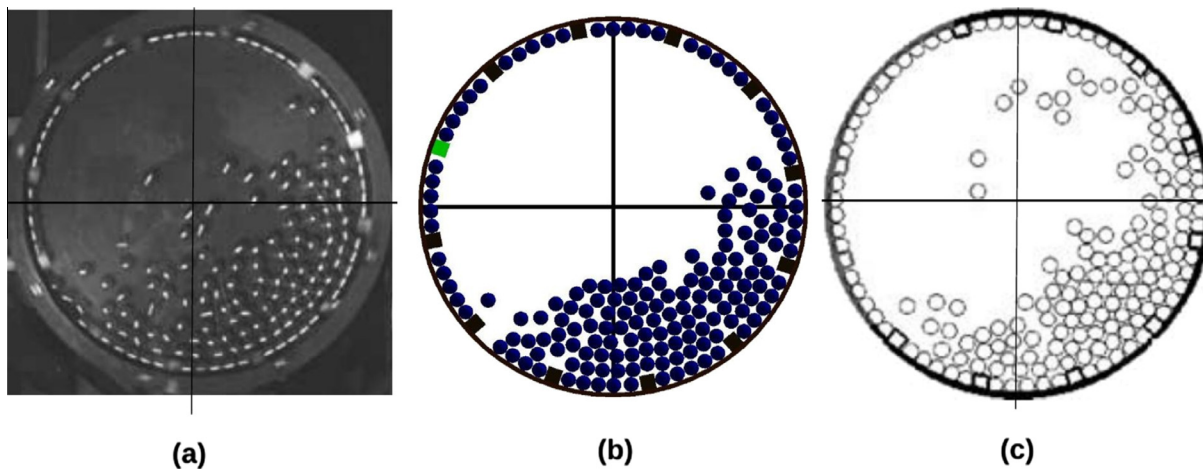
Fig. 20. Power draw for (a) 25% and (b) 35% loading between experiment (Hlungwani et al., 2003), GPU and CPU simulations.



**Fig. 21.** (a) Experiment (Hlungwani et al., 2003), (b) GPU and (c) CPU charge profiles.  $N = 169$  (35% filling),  $rpm = 17.50$  (30% critical speed).



**Fig. 22.** (a) Experiment (Hlungwani et al., 2003), (b) GPU and (c) CPU charge profiles.  $N = 120$  (25% filling),  $rpm = 40.81$  (70% critical speed).



**Fig. 23.** (a) Experiment (Hlungwani et al., 2003), (b) GPU and (c) CPU charge profiles.  $N = 169$  (35% filling),  $rpm = 58.30$  (100% critical speed).

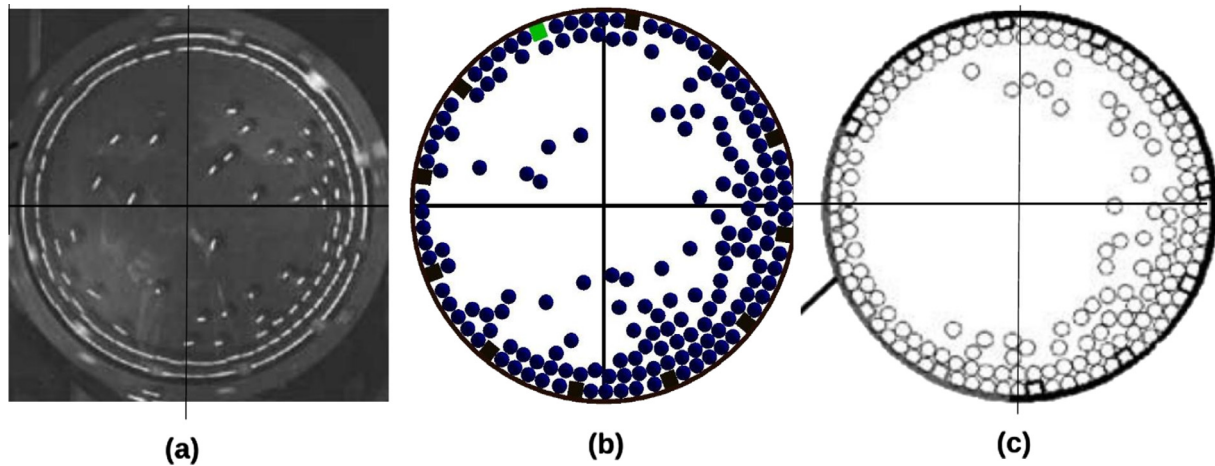


Fig. 24. (a) Experiment (Hlungwani et al., 2003), (b) GPU and (c) CPU charge profiles.  $N = 169$  (35% filling),  $rpm = 93.30$  (160% critical speed).

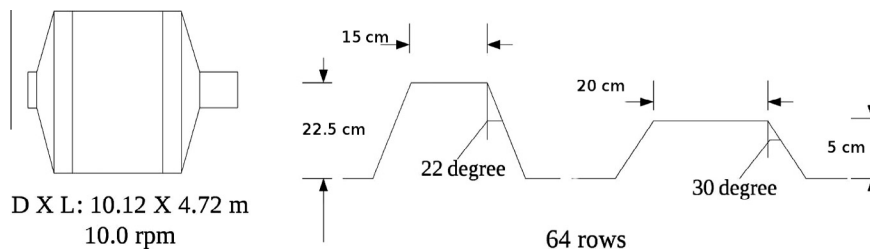


Fig. 25. Lifter design Los Bronces semi-autogenous mill.

(Koski et al., 2011) is 7.1 MW. Since no further information about mill internals was available, typical values of operating parameters for this type of mill was used. It is presumed that the mill would be fitted with 64 rows of high low lifters as shown in Fig. 25.

Equilibrium ball size distribution with top ball size of 12.5 cm was used in the simulation. The ore in the mill charge was approximated as a Gaudin–Schuhmann (Macias-Garcia et al., 2004) distribution with slope 0.6 and top size of 15.0 cm. The combined weight distribution and number of ore and ball particles in the charge mix are shown in Table 7. The aim of this study is to do a qualitative investigation as opposed to a quantitative investigation due to the uncertainties in the plant mill operating data. We investigate how the power consumption varies over revolutions, as well as how the power gets dissipated inside the mill.

Table 8 summarizes the model parameters used in the simulations in Section 3. We tuned the parameters to match the charge profile obtained by Rajamani et al. (2011).

Fig. 26 shows the charge profile in the mill. The charge profile is what one would anticipate, for such large filling of 41%. The charge shoulder is nearly at 2 o'clock and the toe is between 7 and 8 o'clock points on the mill circle. We also see radial segregation with the smallest particles moving to the mill shell as predicted by theory and experiment.

The computed power seems to stabilize after six revolutions as shown in Fig. 27(a). The large number of spheres in the simulation smooth out the energy consumed in collisions per revolution. Hence, accurate dependence of contact parameters on contact velocity is unnecessary here. The computed power consumption is 6.8 MW, which is slightly lower than the experimental value of 7.1 MW. Note, this value is arbitrary as we showed we could match the power consumption exactly by changing the coefficient of restitution. The important observation is how the power

Table 7  
Charge distribution for Los Bronces mill.

| Diameter (cm) | Density (kg/m <sup>3</sup> ) | Weight (%) | Number         |
|---------------|------------------------------|------------|----------------|
| <b>ORE</b>    |                              |            |                |
| 15.0          | 2850                         | 9          | 6320           |
| 12.5          | 2850                         | 9          | 9020           |
| 10.1          | 2850                         | 30         | 81,881         |
| <b>BALLS</b>  |                              |            |                |
| 12.4          | 7800                         | 21         | 9800           |
| 10.0          | 7800                         | 19         | 16,171         |
| 8.7           | 7800                         | 12         | 16,220         |
| <b>TOTAL</b>  |                              |            | <b>139,392</b> |

Table 8  
Model parameters used in simulation of Los Bronces mill.

| Parameter | $K_n$ (N m <sup>-1</sup> ) | $\epsilon_{particle}$ | $\epsilon_{lifter}$ | $K_T$ | $\mu_{particle}$ | $\mu_{lifter}$ | $\Delta t$         |
|-----------|----------------------------|-----------------------|---------------------|-------|------------------|----------------|--------------------|
| GPU       | $1.5 \times 10^6$          | 0.75                  | 0.75                | –     | 0.40             | 0.70           | $2 \times 10^{-5}$ |

consumption varies over revolutions, as well as how this power is dissipated. Fig. 27(b) shows the contribution to power draw by particle to particle collisions, particle to mill shell collisions and particle to lifter collisions. Around 68% of the power draw comes from the lifter since it lifts the majority of the load to the shoulder of the mill and hence excessive forces occur at the points of contact on the lifters. Likewise, around 18% is contributed by the mill shell as it supports the load between two lifters. Thus less than 14% is contributed by particle–particle collision. In addition, as the particle lifter power decreases slightly over revolutions, the mill shell power draw increases slightly. The particle–particle power dissipation remains virtually constant over the revolutions.



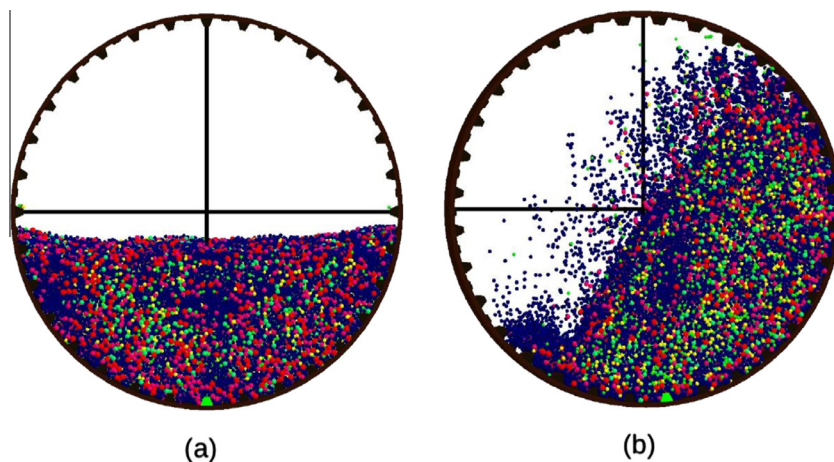


Fig. 26. (a) Initial conditions  $N = 139,392$  (41% filling) and (b) charge profile of Los Bronces mill.

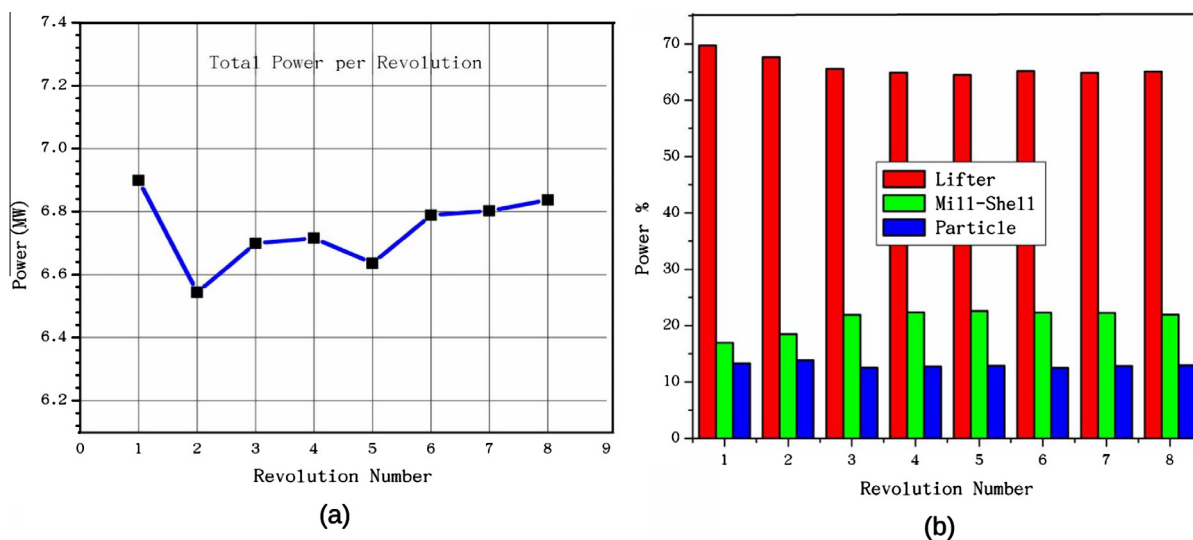


Fig. 27. (a) Total power draw and (b) power distribution over time of Los Bronces mill.

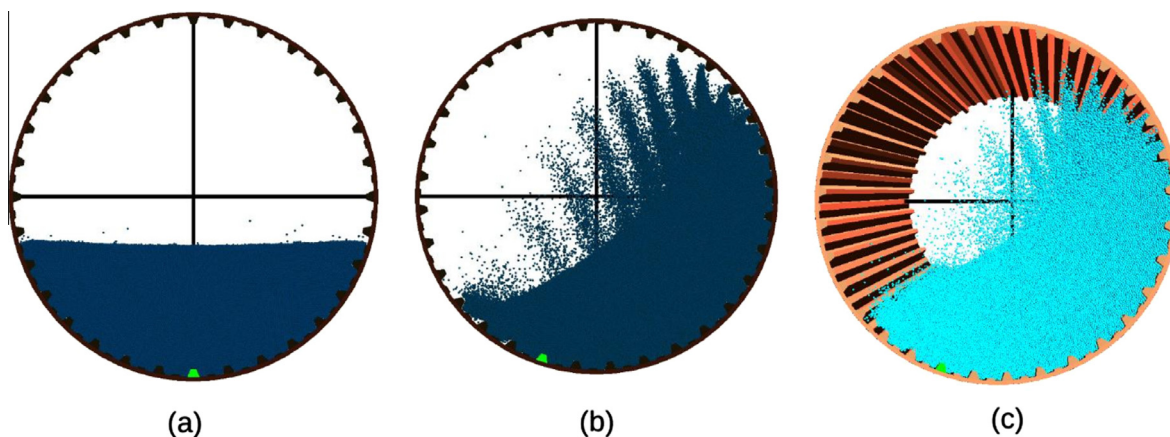


Fig. 28. (a) Initial conditions  $N = 4 \times 10^6$  (35% filling), (b) steady state profile (orthogonal view) and (c) steady state profile (isometric view).

### 3.1. Performance scaling

To gauge the performance of our code we increased the length of the mill to 2800 cm to accommodate four million mono-sized steel balls with a diameter of 6 cm. Fig. 28 shows the charge profile.

The power draft is distributed with 47% to particle–particle collisions, 44% to particle–lifter collisions and 9% to particle mill-shell collisions. The sheer number of particles simulated results in particle–particle collisions consuming the most amount of energy. The GPU compute time for one revolution (6 s) using a time step of  $2 \times 10^{-5}$  with a NVIDIA Kepler GPU is 7 h (12 FPS). Fig. 29 shows

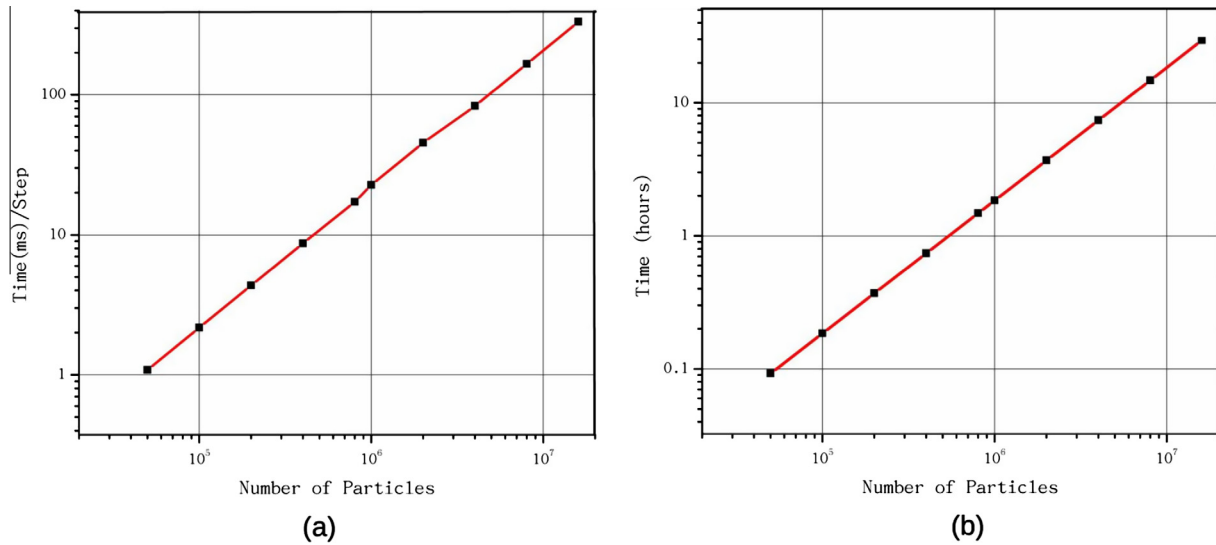


Fig. 29. Scaling of GPU code with number of particles (N).

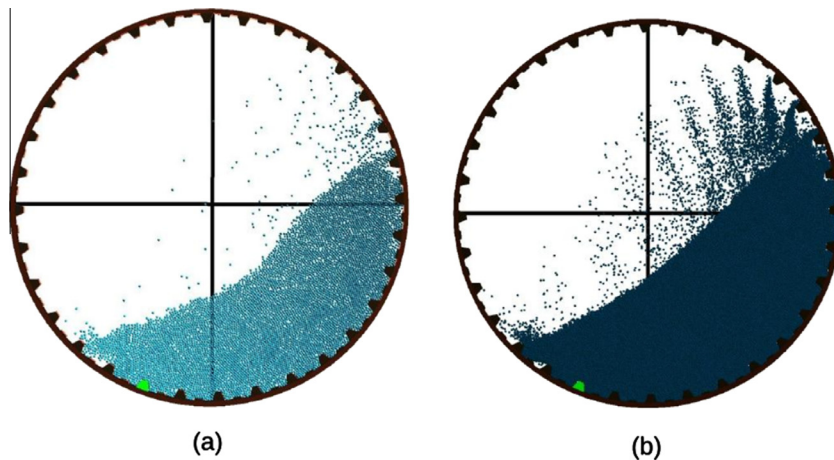


Fig. 30. (a) 2D steady state profile,  $N = 6744$ , (b) Steady state profile for a slice 10% of the length,  $N = 385,534$ .

the scaling of our code with increased number of particles. We observe a trend of linear scaling which is a good indication of the scalability of our code. The GPU compute time for one revolution (6 s) of a simulation consisting of 10 million particles using a time step of  $2 \times 10^{-5}$  with a NVIDIA Kepler GPU will take just 18.5 h with a simulation of 100 million particles taking just over a week.

To show the benefits of a full 3D simulation we performed the same simulation in 2D with the same parameters and obtained a very different charge profile as depicted in Fig. 30(a). Fig. 30(b) depicts the charge profile for a slice of 10% of length. We notice that the pattern is very similar to the full length of the mill. Clearly the effect of the axial direction cannot just be neglected as the 2D case cannot fully reproduce the dynamics. The run-time for the 2D case was 3 min while the run-time for the slice was 44 min. Note that the parallelism of the GPU is not fully exploited in 2D case which has very few particles resulting in lower performance.

#### 4. Conclusions

In this paper we have presented a novel approach for modeling tumbling mills utilizing the GPU architecture. The modular approach of our code allows us to analyze the distribution of power

amongst different collision types in the mill which lends insight that may be exploited to improving the energy efficiency of mills. We achieve a new performance level in DEM modeling of mills by simulating 16 million particles at 3 FPS on a laptop GTX 880 GPU. Our code can handle 1 billion particles using the 12 GB of memory available on a NVIDIA K40 GPU. However such a large simulation should ideally be run with multiple GPUs and is currently under development.

#### Acknowledgments

The first author acknowledges the support of the university of Utah in providing research funds for his stay at the university to do this work. We gratefully acknowledge the support of NVIDIA Corporation for the donation of the GPU used for this research.

#### References

- Alatalo, J., Tano, K., 2010. Comparing experimental measurements of mill lifter deflections with 2D and 3D DEM predictions. In: DEM5 Proceedings, Queen Mary University, London, UK, vol. 1, pp. 194–198.
- Bell, N., Yu, Y., 2005. Particle-based simulation of granular materials. In: Eurographics/ACM SIGGRAPH Symposium on Computer Animation, vol. 25, pp. 29–31.

- Cleary, P.W., 2001. Recent advances in DEM modelling of tumbling mills. *Miner. Eng.* 14, 1295–1319.
- Cleary, P.W., Morrison, R., 2009. Particle methods for modelling in mineral processing. *Int. J. Computat. Fluid Dynam.* 23, 137–146.
- Cleary, P., Sawley, M., 1999. Three-dimensional modeling of industrial granular flows. In: *Proceeding of CFD in the Minerals and Process Industries*. CSIRO, Melbourne, Australia.
- Cleary, P., Sawley, M., 2002. DEM modelling of industrial granular flows: 3D case studies and the effect of particle shape on hopper discharge. *Appl. Math. Model.* 26, 89–111.
- Cundall, P., 1988. Formulation of a three-dimensional distinct element model – part i: a scheme to detect and represent contacts in a system composed of many polyhedral blocks. *Int. J. of Rock Mech* 25, 107–116.
- Cundall, P., Strack, 1979. A discrete numerical model for granular assemblies. *Geotechnique* 29, 47–65.
- Dennis, M., Rajamani, R., 2001. Evolution of the perfect simulator. *Int. Autogen. Semiautogen. Grind. Technol. Proc.* 31, 163–193.
- Favier, J., 2014. Edem code (December). <<http://www.dem-solutions.com>>.
- Govender, N., Wilke, D., Kok, S., 2014. Collision detection of convex polyhedra on the NVIDIA GPU architecture for the discrete element method. *J. Appl. Math. Computat.*, <http://dxdoi.org/10.1016/j.amc.2014.10.013>.
- Govender, N., Wilke, D., Kok, S., Els, R., 2014. Development of a convex polyhedral discrete element simulation framework for NVIDIA Kepler based GPUs. *J. Computat. Appl. Math.* 270, 63–77.
- Harada, T., 2008. GPU Gems 3: Real-time Rigid Body Simulation on GPUs, Vol. 3. Addison-Wesley.
- Herbst, J., Nordell, L., 2001. Optimization of the design of SAG mill internals using high fidelity simulation. In: *International Conference on Autogenous and Semiautogenous Grinding Technology*, vol. 31, pp. 150–164.
- Hlungwani, O., Rikhotso, J., Dong, H., Moys, M., 2003. Further validation of DEM modeling of milling: effects of liner profile and mill speed. *Miner. Eng.* 16, 993–998.
- Hromnik, M., 2013. Masters Thesis: A GPGPU implementation of the discrete element method applied to modeling the dynamic particulate environment inside a tumbling mill, University of Cape Town, <<http://www.uct.ac.za>>.
- Jaelee, S., 2014. PhD Thesis: DEVELOPMENTS IN LARGE SCALE DISCRETE ELEMENT WITH POLYHEDRAL PARTICLES SIMULATIONS, University of Illinois at Urbana-Champaign, <[www.uiuc.edu](http://www.uiuc.edu)>.
- Koski, S., Vanderbeck, J., Eriques, J., 2011. Cerro Verde Concentrator-Four Year Operating HPGRs. In: *Proceeding of the SAG Conference*, Vancouver.
- Langston, P., 2004. Distinct element modelling of non-spherical frictionless particle flow. *Chem. Eng. Sci.* 59, 425–435.
- Longmore, J., Marais, P., Kuttel, M., 2013. Towards realistic and interactive sand simulation: a GPU-based framework. *Powder Technol.* 235, 983–1000.
- Macias-Garcia, A., Eduardo, Cuerda-Correa, M., Diaz-Diez, M., 2004. Application of the Rosin-Rammler and Gates-Gaudin-Schuhmann models to the particle size distribution analysis of agglomerated cork. *Mater. Character.* 52, 159–164.
- Mack, S., Langston, P., Webb, C., York, T., 2011. Experimental validation of polyhedral discrete element model. *Powder Technol.* 214, 431–442.
- Mishra, B., Rajamani, R., 1990. Numerical simulation of charge motion in a ball mill. In: *7th European Symposium on Comminution*. vol. 1, pp. 555–563.
- Mishra, B., Rajamani, R., 1994. Simulation of charge motion in ball mills. Part 1: experimental verifications. *Int. J. Miner. Process* 40, 171–186.
- Mishra, B.K., Rajamani, R., 2001. Three dimensional simulation of plant size SAG mills. In: *International Conference on Autogenous and Semiautogenous Grinding Technology*, vol. 31, pp. 48–57.
- Morrison, R., Cleary, P.W., 2008. Towards a virtual comminution machine. *Miner. Eng.* 21, 770–781.
- Munjiza, A., Andrews, K., 1998. Nbs contact detection algorithm for bodies of similar size. *Int. J. Numer. Meth. Eng.* 43, 131–149.
- Neubauer, G., Radek, C.A., 2014. GPU Based Particle Simulation Framework With Fluid Coupling Ability, NVIDIA GTC 2014, San Jose, USA. <<http://on-demand.gputechconf.com/gtc/2014/poster/pdf/P4143>>.
- Powell, M., 1991. The effect of liner design on the motion of the outer grinding elements in a rotary mill. *Int. J. Miner. Process.* 31, 163–193.
- Rajamani, R., Callahan, S., Schreiner, J., 2011. DEM Simulation of mill charge in 3D via GPU computing. In: *Proceeding of the SAG Conference*, Vancouver.
- Sanders, J., Kandrot, E., 2010. CUDA by Example, Vol. 12. Addison-Wesley.
- Venugopal, R., Rajamani, R., 2001. 3d simulation of charge motion in tumbling mills by the discrete element method. *Powder Technol.* 115, 157–166.
- Walther, J.H., Sbalzarini, F., 2009. Large-scale parallel discrete element simulations of granular flow. *Eng. Computat.* 26, 688–697.
- Zhao, D., Nezami, E., Hashash, Y., Ghaboussi, J., 2006. Three-dimensional discrete element simulation for granular materials. *Comput.-Aided Eng. Computat.: Int. J. Eng. Software* 23, 749–770.